

4.7 K-MEANS CLUSTERING ALGORITHM

DESIGN AND METHODOLOGY :

K-means is one of the simplest unsupervised learning algorithms that solve the well known clustering problem. The procedure follows a simple and easy way to cluster a given data set through a certain number of clusters which is given a priori. The main idea is to define k centroids, one for each cluster. These centroids should be placed in a cunning way, because different location causes different result. So, the better choice is to place them as much far away from each other as possible.

ALGORITHM :

1. For K-number of clusters define K number of centroids as far away from each other as possible.
2. For each object of the data set, the object is placed in the group belonging to the centroid to which it is closest.
3. Recalculation of centroids are done by the formula.

$$\frac{\sum X_k(i, j)}{N_k} \quad [1]$$

Where $X_k(i, j)$ is the objects belonging to k_{th} cluster and N_k is the number of elements in k_{th} cluster.

4. Steps 2 and 3 are repeated until the centroids do not change any further and thus finds the appropriate centroids for k number of clusters.

INCORPORATING NEIGHBORHOOD INFORMATION :

The K-Means Clustering Algorithm provides one of the simplest ways for segmentation but is not as efficient as some of the other algorithms. However, to make it a little bit more efficient, we have incorporated the neighborhood information in the algorithm. The incorporation of neighborhood information allows us to deal with 9 features of each object in the image. The inclusion of nine features instead of just one feature gives better result especially in noisy images. The proposed method gives better result than conventional K-Means Clustering algorithm.

ALGORITHM : K-MEANS NEIGHBORHOOD

1. Read the input image.
2. Take the number of clusters, k as user input.
3. Define the value of centroids for each cluster. It may be taken as user input or maybe randomly selected.
4. For each object of the data set, compare its nine features, which is a 3x3 window around the object, with each centroid. Here, we have used euclidian distance for comparison and placed the object to that particular cluster for which the distance is minimum.

$$D_{i,k} = \frac{1}{n} \sum_n \|x_{i,n} - v_k\|^2 \quad [2]$$

where $D_{i,k}$ is the euclidian distance between i_{th} object and k_{th} centroid, n is the number of neighborhood elements around x_i , v_k is the k_{th} centroid and $x_{i,n}$ is the n_{th} neighborhood object of the object.

5. The centroids for each plane are recalculated by using the formula:

$$v_k = \frac{\sum_{i=1}^N \frac{1}{n} \sum_n x_{i,n}}{N} \quad [3]$$

where k is the number of centroids and N is the total number of objects in the input file.

5. Steps 2 to 4 are repeated until the centroids for each plane do not change any further and thus finds the desired clusters.

SAMPLE CODE :

```
#include<stdio.h>
#include<conio.h>
#include<stdlib.h>
#include<string.h>
#include<math.h>
#define row row_number           //row defines the number of rows in the input image
#define col column_number       //col defines the number of columns in the input image

int mx[row*col];                //matrix storing the pixel values
float set[20][row*col];         //stores the elements belonging to particular cluster. Here we have set the maximum
                                //number of segments as 20.

main()
{
    printf("#### K-MEANS CLUSTERING ####\n\n");
    int i,j,s=0,num,cnt1=0,cnt=1,cntrcnt,p[10],count=1; //cntrcnt is the number of centroids,p[] is the counter to
                                                        //count the elements entering each cluster at each iteration
```

```

float numf,centroid[10],centroid_new[10],d,diff;    //centroid[] stores the values of centroids,centroid_new
                                                    //stores the new computed values of centroids

FILE *f1,*f2;
char c,str1[]="P2\n",str3[]=" 255\n",spc=' ',nl='\n';
f1=fopen("input.txt","r");                        //input.txt stores all the elements of the input image
printf("enter the no of segments: ");              //taking the number of centroid as input from the user
scanf("%d",&cntrcnt);

for(i=0;i<cntrcnt;i++)                            //taking the value of pixels of each centroid as input from the user
{ printf("enter the centroid[%d]: ",i) ; scanf("%f",&centroid[i]); }

while(fgetc(f1))!=EOF)    //taking input from source file and storing it in the matrix mx[]
{
    //The elements are stored in matrix mx[] and the loop is iterated until the end of file is reached
}

cnt=1;
while(cnt==1)
{
    k-means(); //it calculates the clusters corresponding to each centroid and finds the values of new
               //centroids. The value of cnt is changed if all the new found centroids remain the same
}

for(i=0;i<(row*col);i++)    //the segmented image is being created
{
    for(j=0;j<cntrcnt;j++)
    {
        if(mx[i] belongs to set[j][k]) //if the object belongs to a cluster, it is replaced by the
        {                               // centroid of that cluster
            mx[i]=int(centroid[j]);
            s=1;
        }
        if(s==1)
            break;
    }
}

f2=fopen("segmented_image.txt","w");

for(i=0;i<row*col;i++)    //the values of objects after clustering is written in the text file
{
    //write each values of matrix mx[i] to the text file
}

fclose(f2);
fclose(f1);
getch();
}

```

RESULTS :

ORIGINAL IMAGE			SEGMENTED IMAGE					
			K=5			K=7		
								
								
								
								
								